CE5507 Modelacion Hardware Software con Orientacion a Objetos

Taller 5 Introducción a Git

Profesor:
Marco Hernández Vásquez

Estudiante:
Andrés Molina Redondo | 2020129522
Kevin Esteban Chinchilla Rodríguez | 2021101242

Primer Semestre

Año 2026

Introducción a Git

Parte 0: Investigación

1. **Control de versiones**

El control de versiones es un sistema que rastrea, gestiona y permite la reversión de cambios de archivos a lo largo del tiempo.

2. **Repositorio**

Un repositorio es un espacio donde se almacenan archivos, de manera común, código fuente. Consta de historial de cambios y archivos asociados a un proyecto.

3. **Commit**

Un commit es una acción de guardar los cambios realizados en los archivos almacenados en un repositorio. Además, se puede incluir un mensaje describiendo los cambios realizados.

4. **Conflicto (contexto de versionamiento)**

Un conflicto es cuando ocurren dos o más cambios por diferentes personas en las mismas líneas de código, generando una incompatibilidad. Se deben resolver los conflictos comparando versiones manualmente.

5. **Rama (branch)**

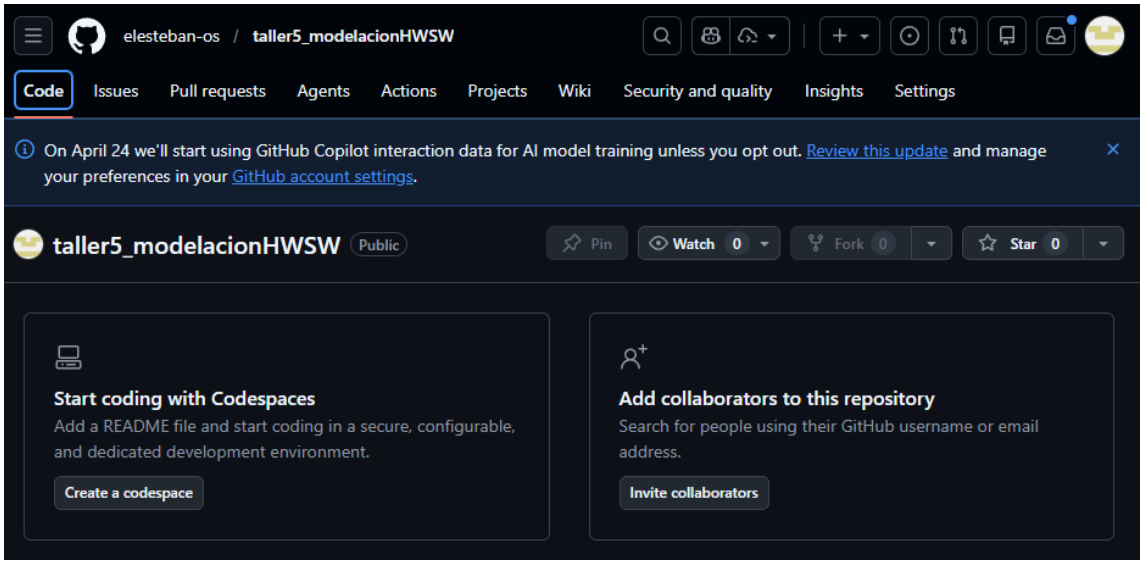
Una rama es una versión paralela del proyecto donde se puede modificar y desarrollar nuevas funcionalidades sin que afecte la versión base del proyecto.

6. **Archivo Markdown (.md)**

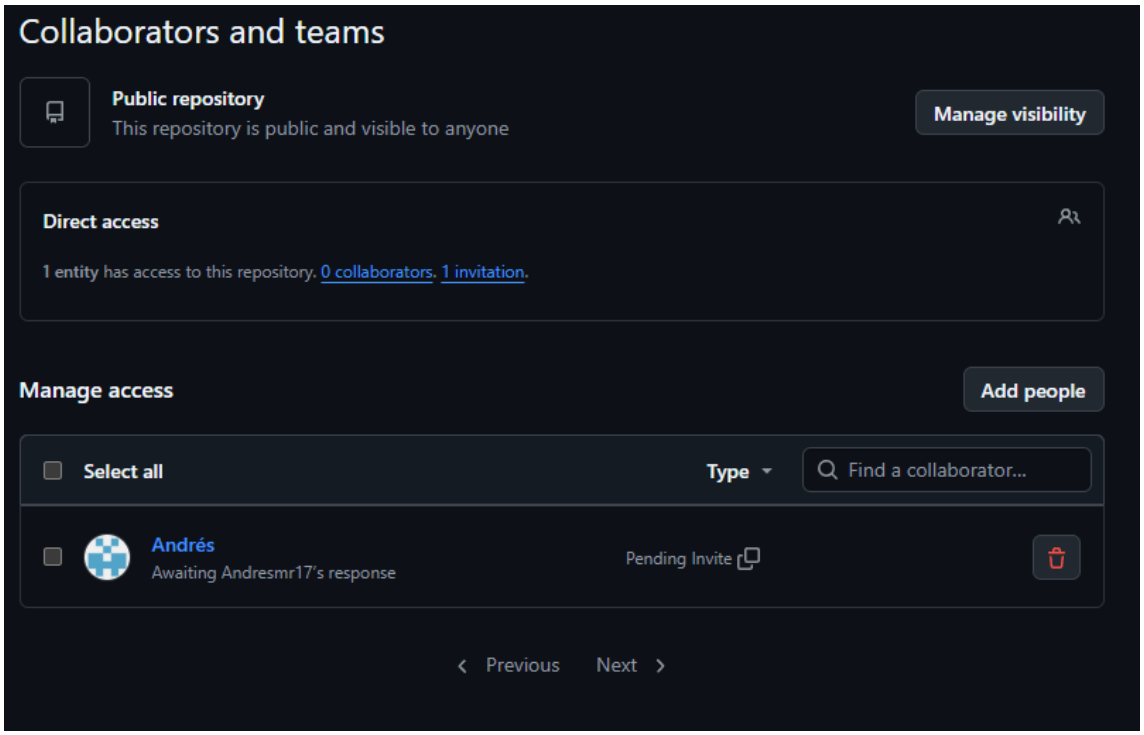
Un archivo Markdown es un tipo de documento de marcado simple donde se puede formatear texto usando símbolos. Es un formato utilizado para documentación o archivos README.

Parte 1: Creación del repositorio

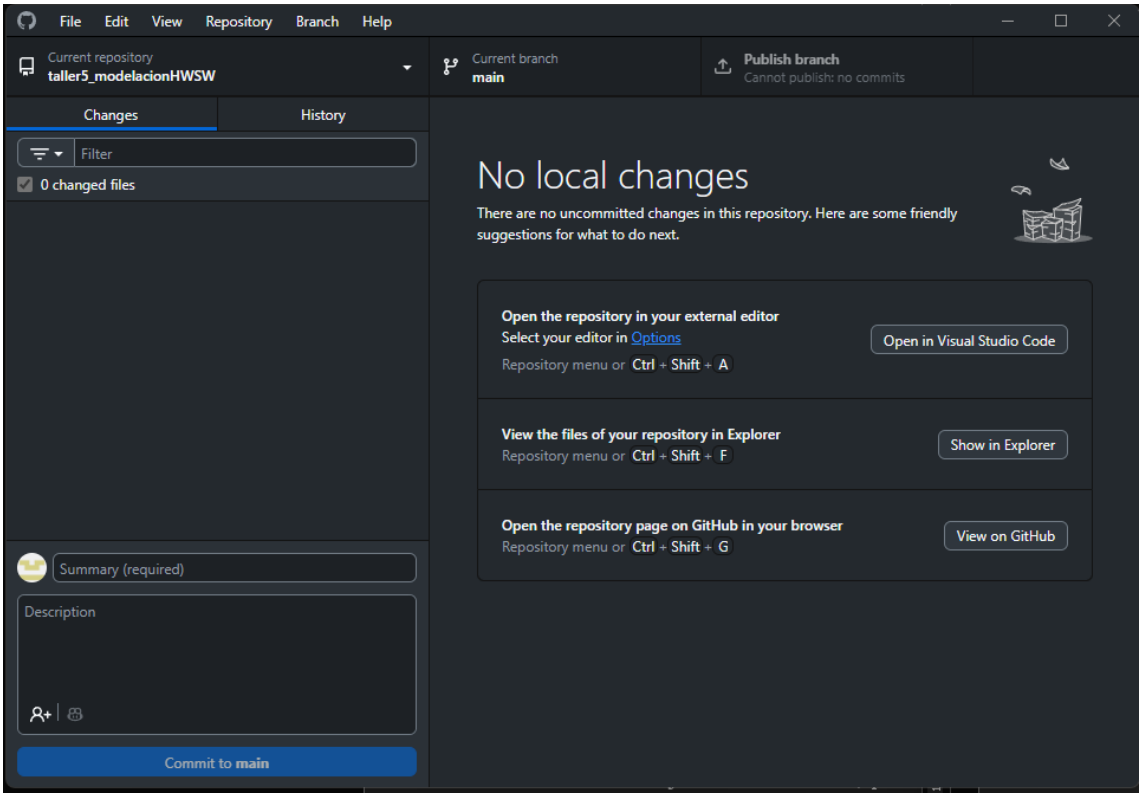
1. **Uno de los integrantes debe crear un repositorio en GitHub.**



2. **Debe agregar a su compañero como colaborador.**

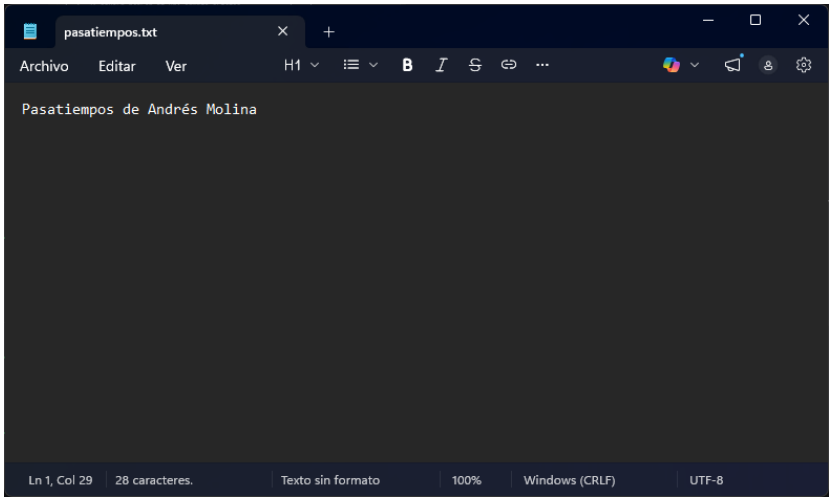


3. **Ambos deben clonar el repositorio utilizando GitHub Desktop.**



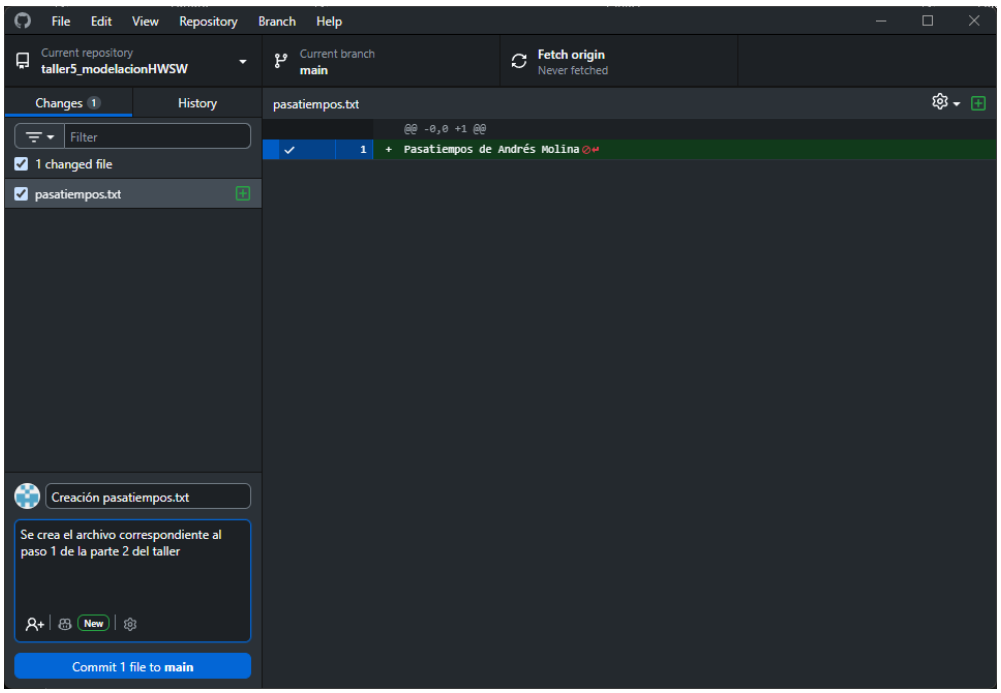
Parte 2: Primeros cambios

1. **Uno de los integrantes debe crear un archivo llamado pasatiempos.txt.**

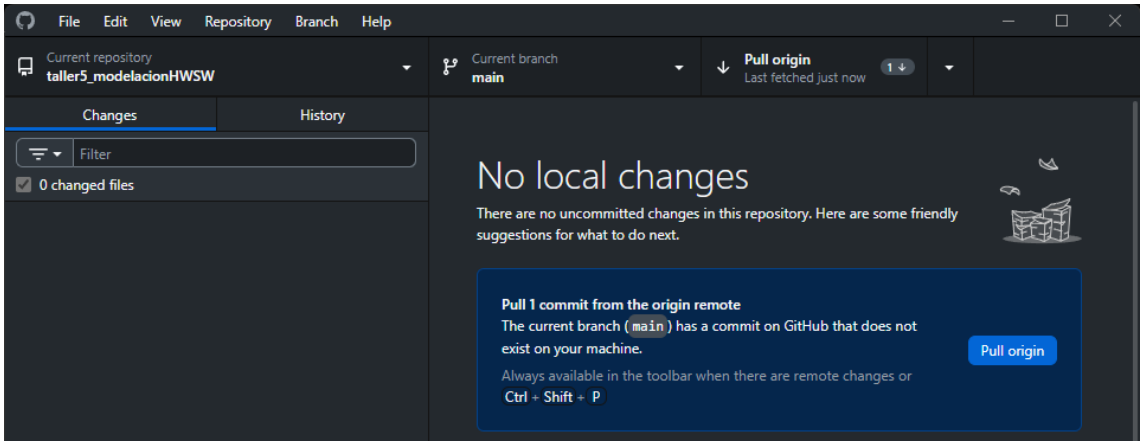


2. **En este archivo deben colocar en la primera línea: Pasatiempos de Nombre Integrante 1 (no incluir al segundo integrante).**

3. **Desde GitHub Desktop: a. Visualizar los cambios realizados b. Realizar un commit con un mensaje descriptivo c. Push los cambios al repositorio**

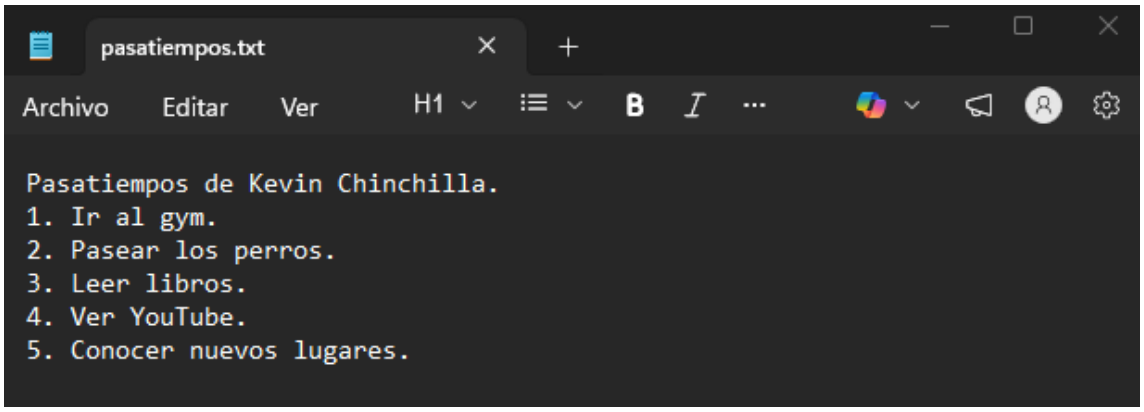


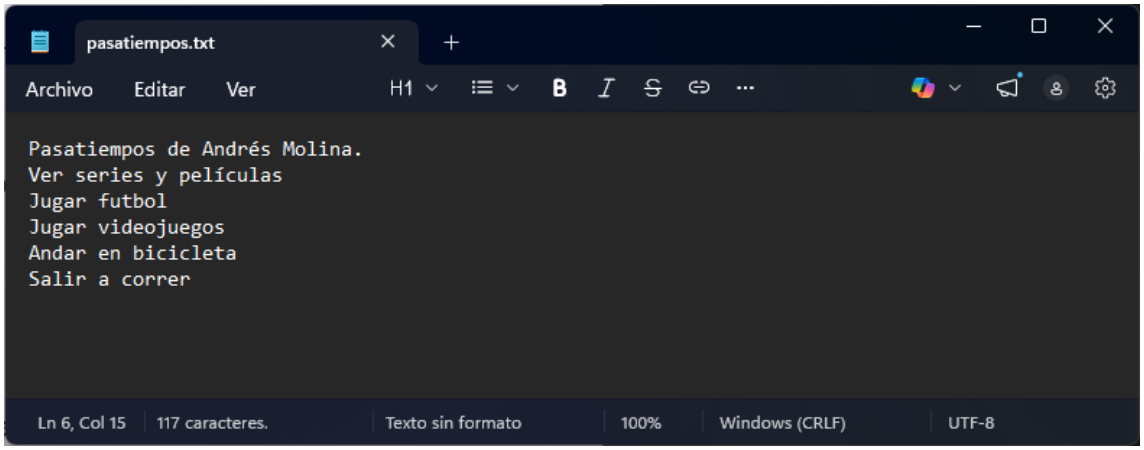
4. **El otro integrante debe hacer un pull del documento.**



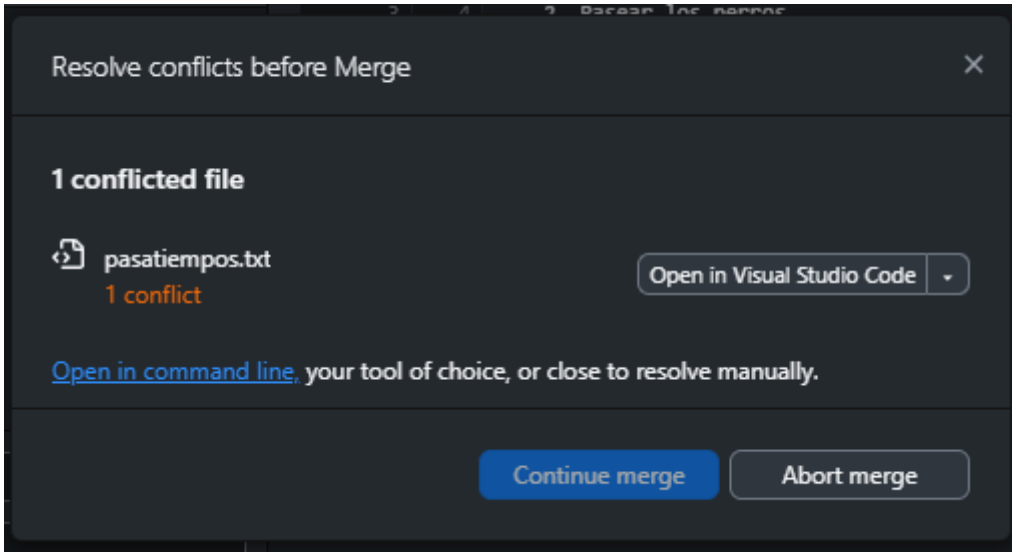
Parte 3: Trabajo en paralelo (conflictos)

1. **Importante: ambos deben hacer los cambios indicados sin realizar pull previamente, para asegurar la generación del conflicto.**
2. **El segundo integrante debe modificar la primera línea para que resulte: Pasatiempos de Nombre Integrante 1, Nombre Integrante 2.**
3. **Ambos integrantes deben agregar sus pasatiempos en el archivo pasatiempos.txt. Colocarlos de forma creativa en el archivo.**
4. **Guardar los cambios localmente.**

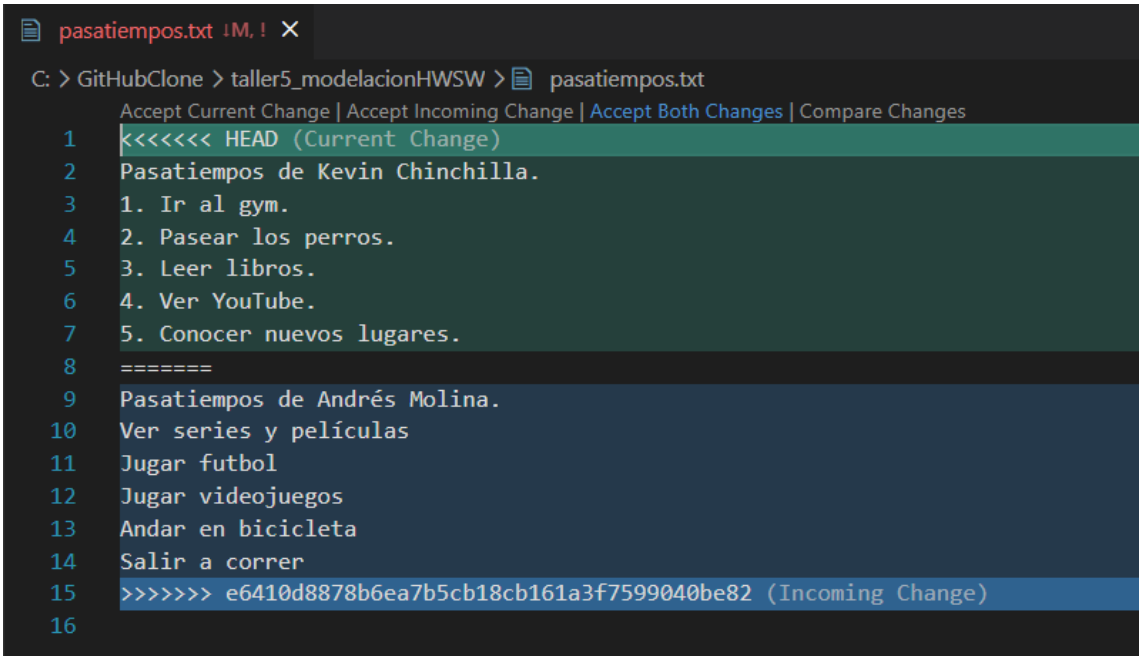




5. **Uno de los integrantes debe: a. Hacer commit b. Push los cambios**
6. **El otro integrante debe: Intentar hacer push de sus cambios.**

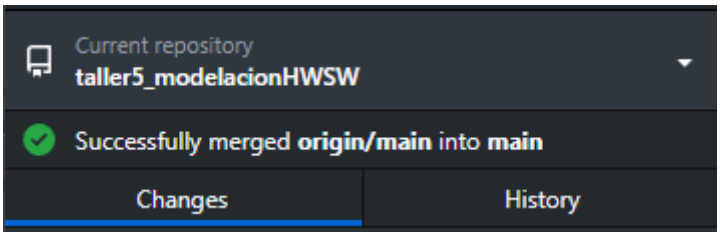


7. **Resolver el conflicto utilizando las herramientas de GitHub Desktop.**



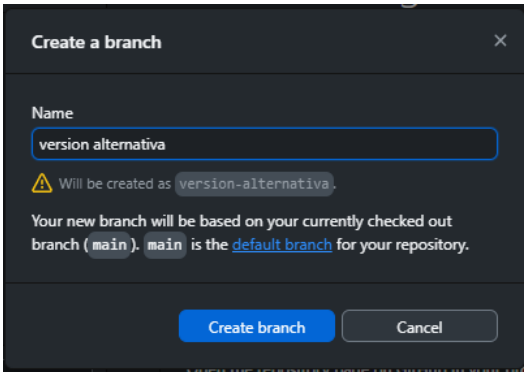
Se aceptaron ambos cambios.

8. **Confirmar los cambios y publicar la versión final.**

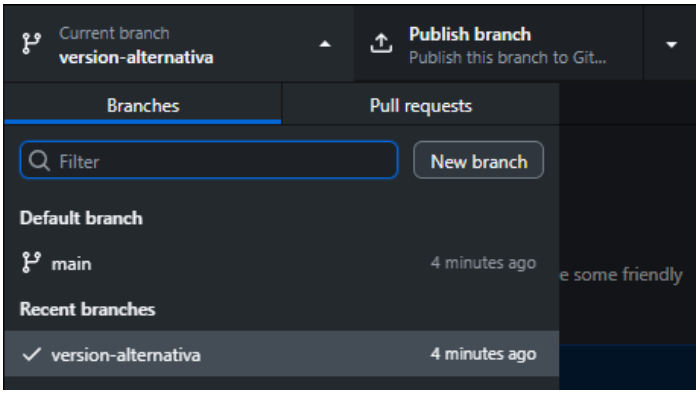


Parte 4: Uso de ramas (branches)

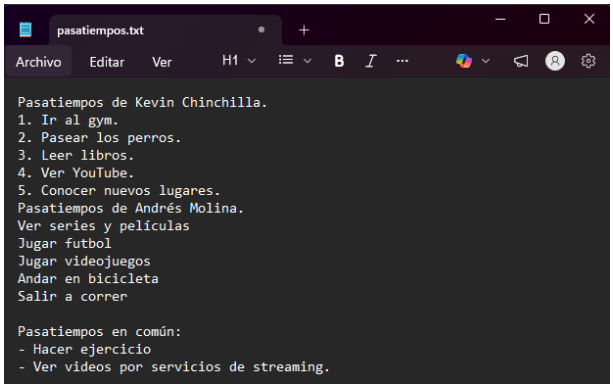
1. **Crear una nueva rama llamada version-alternativa.**



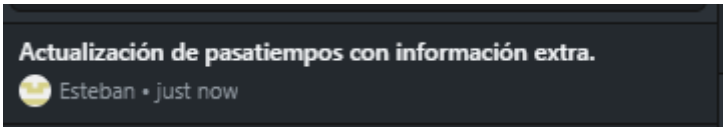
2. **Cambiar a esa rama desde GitHub Desktop.**



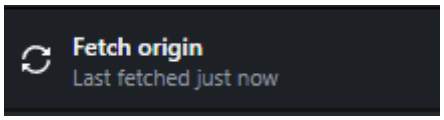
3. **Modificar el archivo pasatiempos.txt (agregar al final los que tienen en común).**



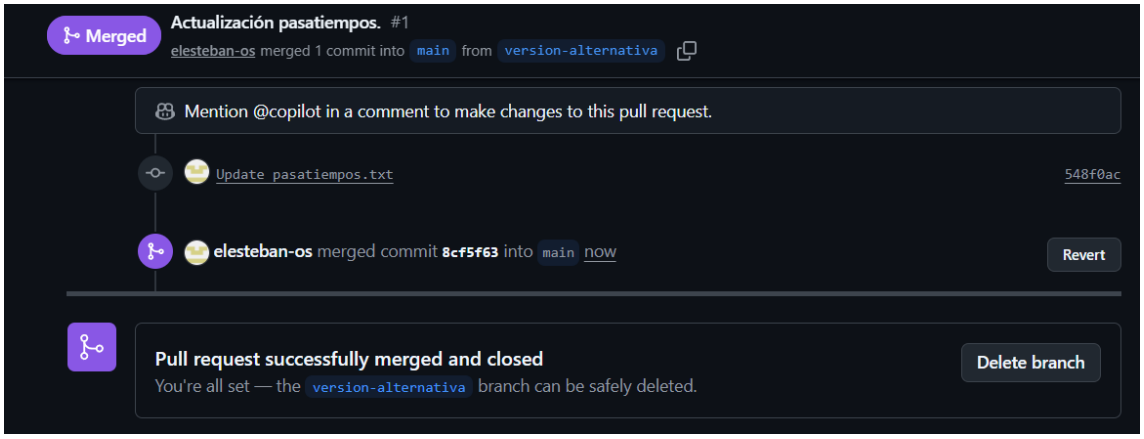
4. **Realizar commit de los cambios.**



5. **Push la rama.**



6. **Crear un Pull Request y realizar el merge hacia la rama principal.**

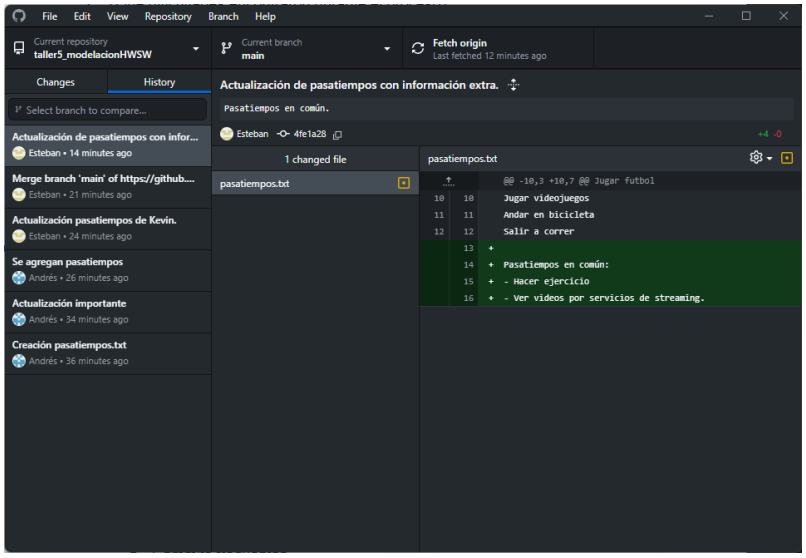


Parte 5: Exploración del historial

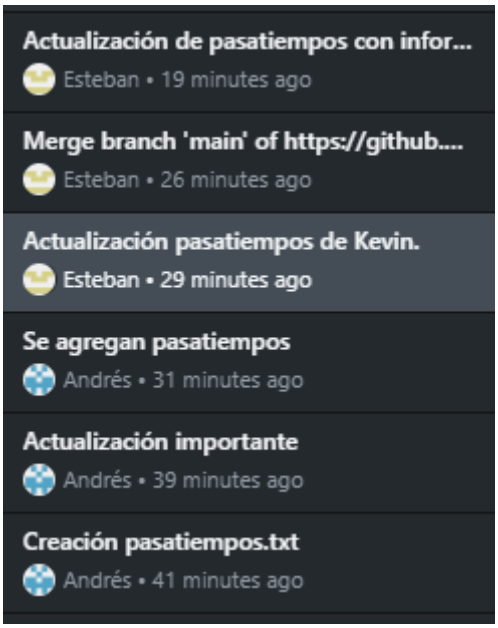
1. **Revisar el historial de commits en GitHub Desktop o en GitHub.**

2. **Identificar:**

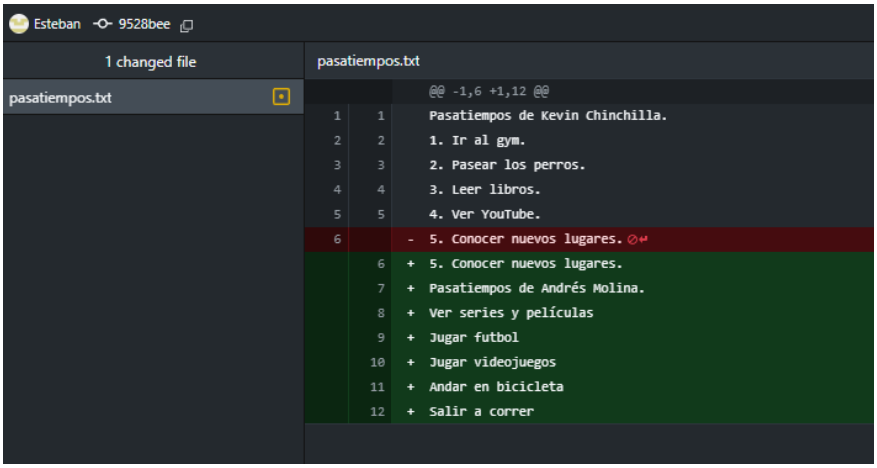
a. Cambios realizados



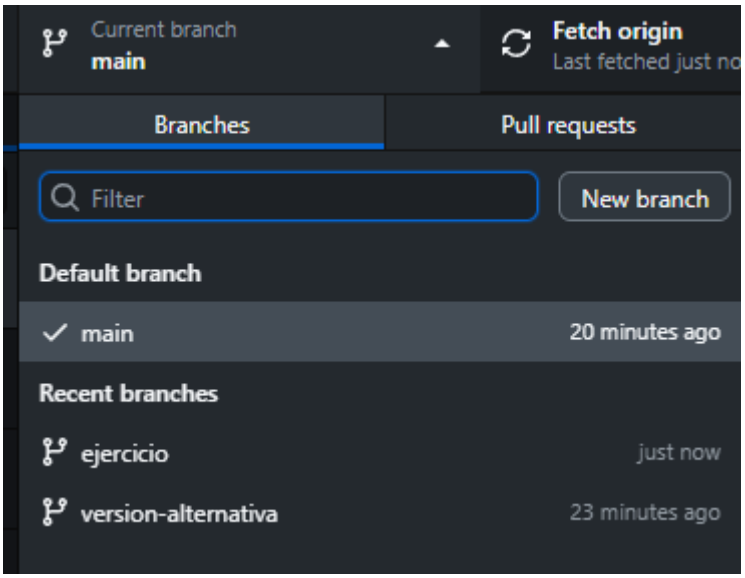
b. Autor de cada cambio



c. Resolución del conflicto



d. Uso de ramas



Parte 6: Reflexión

1. ¿Qué ventajas tiene el uso de control de versiones?

La principal ventaja es que se puede ver todo el historial de cambios y saber exactamente qué se modificó, cuándo y quién lo hizo. También es muy útil porque si algo se borra, se puede volver a una versión anterior sin perder todo el trabajo. Además, permite que varias personas trabajen en el mismo proyecto sin estar modificando los mismos archivos.

2. ¿Qué dificultades encontraron durante el proceso?

El único error forzado fue al modificar el archivo de los pasatiempos, pero en general al ser una herramienta que hemos tenido que estar en constante uso debido a la carrera de Ingeniería en Computadores ya estábamos acostumbrados al uso de esta entonces no tuvimos ninguna dificultad.

3. ¿Qué ocurrió durante el conflicto y cómo lo resolvieron?

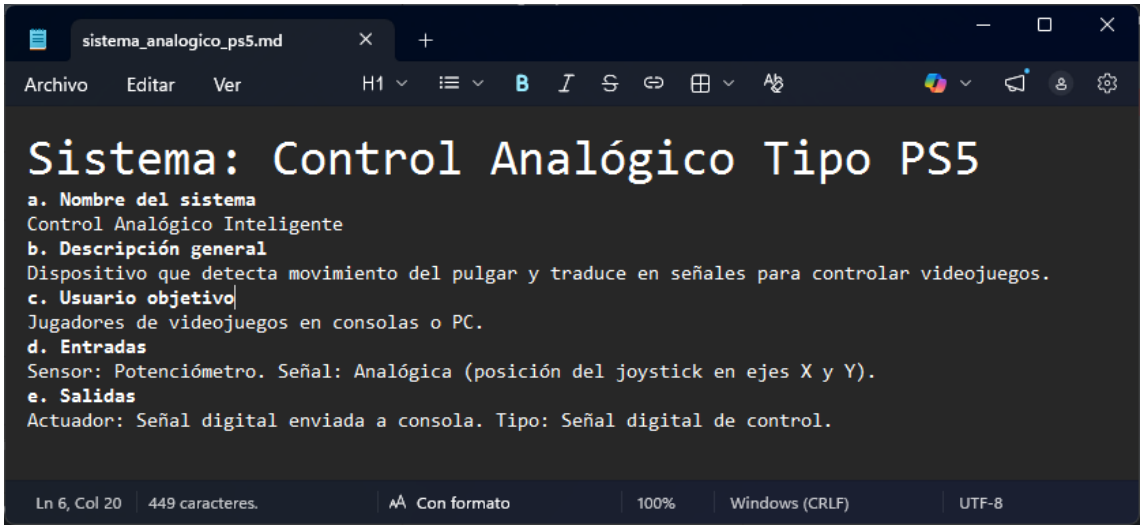
Los dos modificamos el mismo archivo sin hacer pull primero, entonces cuando uno intentó hacer push, Git detectó que había cambios incompatibles en las mismas líneas. GitHub Desktop mostró el conflicto marcando las dos versiones en el archivo. Lo resolvimos aceptando ambos cambios desde GitHub Desktop y luego se completó el merge.

4. ¿Qué utilidad tienen las ramas en un proyecto real?

Las ramas son bastante útiles porque permiten trabajar en una funcionalidad nueva sin afectar el código principal. Por ejemplo, si se está probando algo que no se tiene certeza si va a funcionar, se puede implementar en una rama aparte y si sale bien se une al main. Así el proyecto principal siempre se mantiene estable.

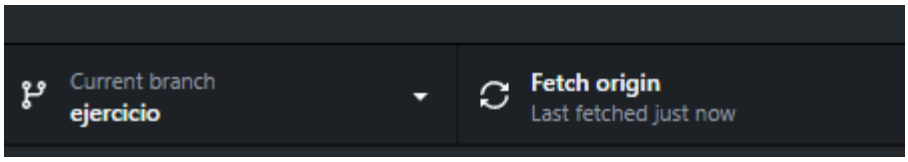
Ejercicio:

1. **Cree un archivo markdown (nombre a decisión del grupo). Deben proponer un sistema interactivo sencillo. Al menos deben incluir: Nombre del sistema**
   a. Descripción general
   b. Usuario objetivo
   c. Entradas (si aplica: el sensor y el tipo de señal)
   d. Salidas (si aplica: el actuador y el tipo de señal)

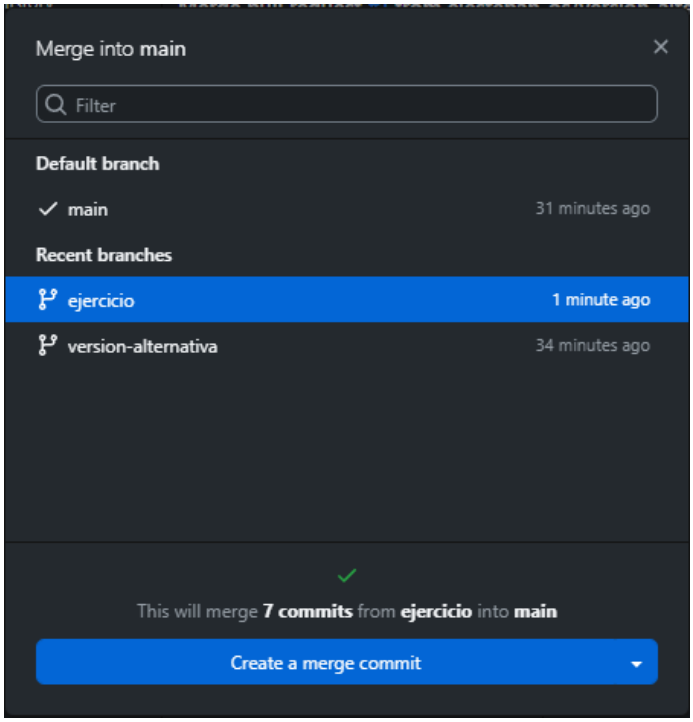


2. **Ambos integrantes deben trabajar sobre el archivo, realizando cambios en paralelo.**
3. **Durante el desarrollo deben utilizar:**
   a. Commits frecuentes

   b. Al menos una rama (branch)



c. Un proceso de merge



4. El historial del repositorio debe evidenciar:

a. Aportes de ambos integrantes
b. Evolución del sistema propuesto
c. Integración de cambios mediante ramas